

Math 329: Continuous Optimization Homework 1 Group V

Álvaro Acedo Blanco, Tahi Govan, Hafidz Arsha Kabira

Solution 1.

We aim to show that the function

$$f_\lambda(\theta) = \sum_{i=1}^m \phi(s_i \langle x_i, \theta \rangle) + \frac{\lambda}{2} \|\theta\|^2$$

is λ -strongly convex and possesses exactly one unique global minimum.

By definition, $f_\lambda(\theta)$ is λ -strongly convex if and only if the following function is convex:

$$g(\theta) = f_\lambda(\theta) - \frac{\lambda}{2} \|\theta\|^2 = \sum_{i=1}^m \phi(s_i \langle x_i, \theta \rangle)$$

The piecewise function $\phi(z)$ is continuously differentiable (C^1) on $\mathbb{R}$, with its derivative given by:

$$\phi'(z) = \begin{cases} 0, & z \leq -1 \\ 1+z, & -1 \leq z \leq 0 \\ 1, & z \geq 0 \end{cases}$$

Since ϕ' is continuous and non-decreasing across all intervals on $\mathbb{R}$, ϕ is convex by Lemma 4.22

For each $i \in \{1, \dots, m\}$, the mapping $\theta \mapsto s_i \langle x_i, \theta \rangle$ is an affine transformation. The composition of a convex function (ϕ) with an affine mapping is convex. Since $g(\theta)$ is a sum of such convex functions, $g(\theta)$ is also convex.

Because $g(\theta)$ is convex, $f_\lambda(\theta)$ is λ -strongly convex, by Definition 4.5. By Theorem 4.28., every strongly convex function defined on a convex set admits a unique global minimum.

Thus, the set of critical points contains exactly one element, which corresponds to the unique global minimum.

Solution 2.

The regularized objective is $f_\lambda(\theta) = \sum_{i=1}^m \phi(s_i \langle x_i, \theta \rangle) + \frac{\lambda}{2} \|\theta\|^2$. Differentiating term by term:

$$\nabla f_\lambda(\theta) = \nabla \left[\sum_{i=1}^m \phi(s_i \langle x_i, \theta \rangle) \right] + \nabla \left[\frac{\lambda}{2} \|\theta\|^2 \right]$$

For the data term, let $z_i(\theta) = s_i \langle x_i, \theta \rangle$. By the chain rule, and since z_i is linear in θ with $\nabla_\theta z_i(\theta) = s_i x_i$:

$$\nabla_\theta \phi(z_i(\theta)) = \phi'(z_i(\theta)) \cdot \nabla_\theta z_i(\theta) = \phi'(s_i \langle x_i, \theta \rangle) s_i x_i$$

Summing over i :

$$\nabla_\theta \sum_{i=1}^m \phi(s_i \langle x_i, \theta \rangle) = \sum_{i=1}^m s_i \phi'(s_i \langle x_i, \theta \rangle) x_i$$

For the regularizer, using $\|\theta\|^2 = \theta^\top \theta$:

$$\nabla_{\theta} \left(\frac{\lambda}{2} \|\theta\|^2 \right) = \lambda \theta$$

Combining both terms gives the full gradient:

$$\nabla f_{\lambda}(\theta) = \sum_{i=1}^m s_i \phi'(s_i \langle x_i, \theta \rangle) x_i + \lambda \theta$$

In matrix form, with $X = [x_1, \dots, x_m] \in \mathbb{R}^{(d+1) \times m}$, $s = (s_1, \dots, s_m)^\top$ and $\odot$ the entrywise product, let $z = s \odot (X^\top \theta) \in \mathbb{R}^m$. Then

$$f_{\lambda}(\theta) = \sum_{i=1}^m \phi(z_i) + \frac{\lambda}{2} \|\theta\|^2, \quad \nabla f_{\lambda}(\theta) = X(s \odot \phi'(z)) + \lambda \theta,$$

where ϕ and ϕ' are applied entrywise. This is the form used in the vectorized implementation: one product $X^\top \theta$ for f_{λ} , and one more product Xw with $w = s \odot \phi'(z)$ for ∇f_{λ} .

Numerical agreement. We evaluated both implementations at five random points with each $\theta \sim \mathcal{U}([-0.01, 0.01]^{785})$ (seed fixed at 42), and computed the relative errors $|f_{\text{loop}} - f_{\text{vec}}|/|f_{\text{loop}}|$ and $\|\nabla f_{\text{loop}} - \nabla f_{\text{vec}}\|/\|\nabla f_{\text{loop}}\|$. For the non-vectorized baseline, we implemented the dot product with a manual for-loop rather than numpy's `np.dot`, to emphasize the performance gain from vectorization. The largest were $6.43 \cdot 10^{-15}$ and $5.8 \cdot 10^{-14}$, a small multiple of machine precision. The remaining difference comes from summing in a different order, so the two implementations agree up to numerical precision.

Run time. We timed both versions on the same inputs with Python's `timeit` and kept the fastest of 5 calls. The vectorized f_{λ} took 2.1 ms, against 1.85 s for the for-loop, so it is about 800 – 1000 times faster. The gradient improved slightly more, from 3.81 s to 4.8 ms, a speedup of about 800.

Comment. The loop version handles the $m(d+1) \approx 10^7$ multiplications one at a time in Python, while the vectorized version computes them all at once as matrix-vector products in NumPy, which is much faster. Since gradient descent evaluates f_{λ} and ∇f_{λ} at every iteration, we use the vectorized versions from here on.

Solution 3.

We took $\theta_0 \sim \mathcal{U}([-0.01, 0.01]^{785})$ (seed fixed at 42) and a random unit direction $v = u/\|u\|$ with $u \sim \mathcal{N}(0, I)$. Let g be the gradient returned by our code at θ . For 101 log-spaced values of $t \in [10^{-8}, 1]$ we computed

$$E(t) = |f_{\lambda}(\theta + tv) - f_{\lambda}(\theta) - t\langle v, g \rangle|.$$

Slope. For $10^{-7} \lesssim t \leq 1$ the curve follows the t^2 line, and a least-squares fit on $[10^{-6}, 10^{-2}]$, chosen to stay clear of the floating-point error below $t \approx 10^{-7}$, gives a slope of 2.000. Below $t \approx 10^{-7}$ it flattens at about 10^{-12} because of round-off.

Does it support correctness? Yes. By Taylor expansion,

$$E(t) = |t\langle v, \nabla f_{\lambda}(\theta) - g \rangle + O(t^2)|.$$

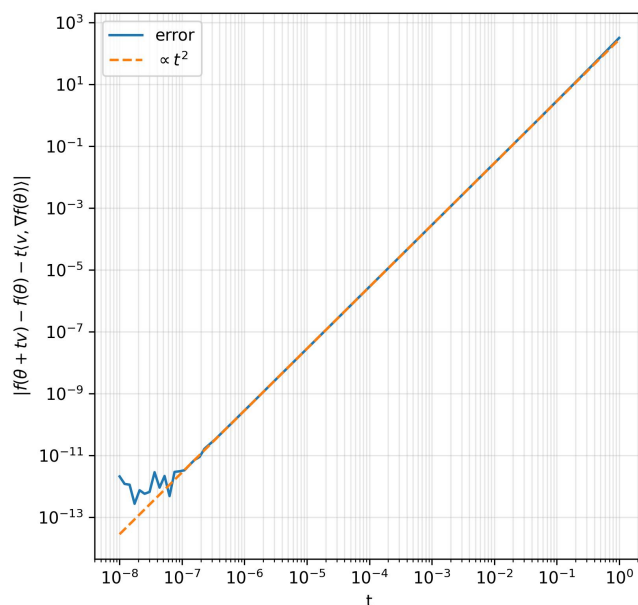


Figure 1: $E(t)$ on log-log axes, with a dashed reference line $\propto t^2$.

A wrong $\langle v, g \rangle$ would leave a linear term that dominates for small t , giving slope 1. The observed slope of 2 means $\langle v, g \rangle$ matches the directional derivative $\langle v, \nabla f_\lambda(\theta) \rangle$.

Does it prove correctness? No. We tested one θ and one v , so an error orthogonal to v , or one that only appears at other points, would go unnoticed. The test also compares g only with our own code for f_λ . If that code were wrong (say, with the wrong ϕ), a gradient of that wrong function would still pass.

Solution 4.

We initialized θ_0 by randomly sampling each theta value from a uniform distribution $\theta_0 \sim \mathcal{U}([-0.01, 0.01]^{785})$ (seed fixed at 42). We ran our GD algorithm with fixed step size $\eta = 10^{-6}$ with the given stopping tolerance of $\|\nabla f_\lambda(\theta_k)\| \leq 10^{-3} \|\nabla f_\lambda(\theta_0)\|$ and a max run time of 3 minutes (using the Python time module). This step size was chosen to sit close to the theoretically safe value $1/L \approx 1.26 \times 10^{-6}$ derived in Question 6.

Before computing question 6, We initially selected $\eta = 10^{-3}$ by trial and error, as opposed to smaller values that would take longer to converge, or larger values that would cause the gradients to oscillate or explode; this run converged to the tolerance in 164 iterations. Choosing instead $\eta \approx 1/L$, as motivated by Theorem 4.31 in Question 6, produces a visibly smoother, more gradual convergence trajectory (seen in question 5) with smaller objective value for f_λ , though at a considerably slower rate, consistent with the very large condition number $\kappa \approx 1.59 \times 10^8$ computed there. Despite this large gap in convergence speed, we found the resulting classifiers to perform comparably at the classification stage (Question 7), with $\eta = 10^{-3}$ yielding a somewhat higher (but still low) test error rate than $\eta = 10^{-6}$. This is consistent with the discussion in Question 6: $1/L$ guarantees convergence but is a conservative, worst-case-safe choice, while a larger empirical step size reaches a

comparably good solution far more quickly.

Solution 5.

We chose a logarithmic vertical axis for the graph because both $f_\lambda(\theta_k)$ and $\|\nabla f_\lambda(\theta_k)\|$ decrease by roughly two orders of magnitude over the run and the linear axis compressed the trajectory into a narrow band near zero, obscuring the convergence behavior making the logarithmic choice more interpretable.

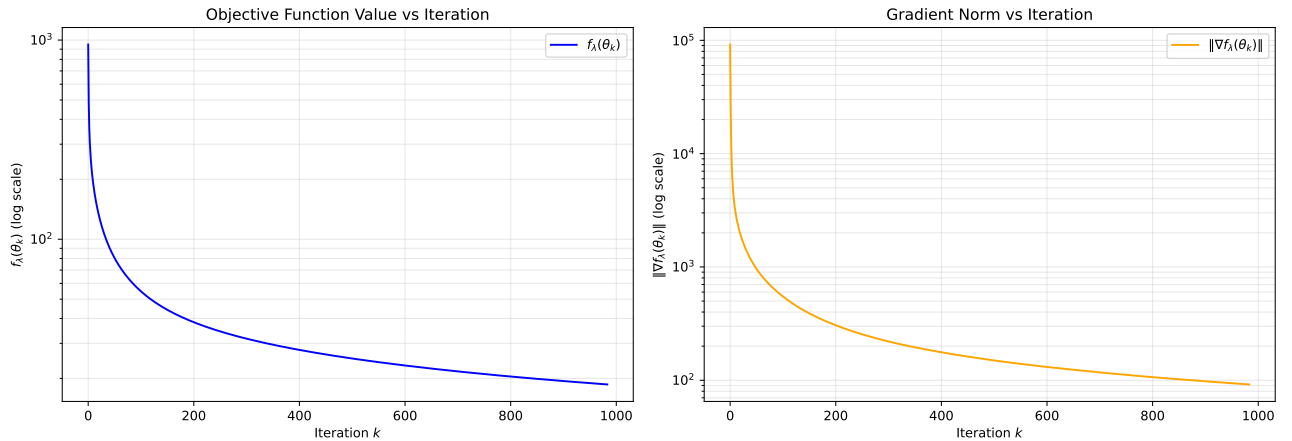


Figure 2: Plot of $f_\lambda(\theta_k)$ and $\|\nabla f_\lambda(\theta_k)\|$ against iterations over the GD algorithm plotted with vertical log.

Solution 6.

The objective $f_\lambda(\theta) = \sum_{i=1}^m \phi(s_i \langle x_i, \theta \rangle) + \frac{\lambda}{2} \|\theta\|^2$ has L -Lipschitz continuous gradient and is μ -strongly convex, so we invoke Theorem 4.31, which guarantees at least linear convergence of fixed-step gradient descent, using step size $\eta = 1/L$, towards the minimizer.

To obtain L , we bound the Hessian of f_λ . From Solution 2, $\phi''(z) \in [0, 1]$ for all z . Each loss term's Hessian is then, by the chain rule,

$$\nabla^2 [\phi(s_i \langle x_i, \theta \rangle)] = \phi''(s_i \langle x_i, \theta \rangle) x_i x_i^\top,$$

and since $\phi''(\cdot) \in [0, 1]$ and $x_i x_i^\top \succeq 0$, summing over i gives

$$0 \preceq \nabla^2 \left(\sum_{i=1}^m \phi(s_i \langle x_i, \theta \rangle) \right) \preceq X^\top X$$

for all θ , where X is the data matrix with rows $x_i^\top$. Adding the regularizer's exact Hessian λI and invoking Theorem 3.3 (a and b) gives

$$\lambda I \preceq \nabla^2 f_\lambda(\theta) \preceq (\|X\|_2^2 + \lambda) I, \quad \forall \theta \implies \mu = \lambda, \quad L = \|X\|_2^2 + \lambda.$$

Computing this on the training data gives $\|X\|_2^2 \approx 891.23$, hence

$$L \approx 7.94 \times 10^5, \quad \mu = \lambda = 0.005, \quad \kappa = \frac{L}{\mu} \approx 1.59 \times 10^8.$$

Theorem 4.31 applies at step size $\eta = 1/L \approx 1.26 \times 10^{-6}$, giving guaranteed linear convergence with at the following rate per iteration

$$1 - \frac{1}{\kappa} \approx 1 - 6.3 \times 10^{-9},$$

a rate so close to 1 that meaningful progress under this exact step would require far more iterations than we could run in 3 minutes.

Although the run plotted in Question 5 uses $\eta = 10^{-6}$, we also evaluated $\eta = 10^{-3}$ due to faster convergence. This step size is almost 10^3 times larger than $1/L$, so Theorem 4.31 does not strictly apply to our chosen step size; nonetheless, at $k = 82$ the theorem's rate at $\eta = 1/L$ predicts $f(\theta_k) - f(\theta^*) \leq \left(1 - \frac{1}{\kappa}\right)^k (f(\theta_0) - f(\theta^*)) \approx 9.87 \times 10^3$, while our observed gap was $\approx 1.29 \times 10^3$, which is about $7.7\times$ smaller, and our run converged to the stopping tolerance in just 164 iterations overall. This confirms $1/L$ is a *sufficient*, conservative step size for at least linear convergence, not a *necessary* one, our run converges markedly faster than this guaranteed rate.

Solution 7.

The classifier achieves very low error on both sets (train: 0.063%, test: 0.047%), with the test error slightly lower than the train error. This indicates the strong generalization of the model. Note that distinguishing handwritten 0s from 1s is a comparatively easy task, and the model shows no sign of overfitting to the training data. The near identical test performance (even slightly better than train performance) is consistent with the strong convexity of f_λ , which guarantees a unique global minimizer, gradient descent therefore converges to a well-defined, stable solution rather than an arbitrary local minimum.

Solution 8.

We have learned that f_λ is used instead of f because if f is convex, then f_λ is strongly convex, which has nice properties for gradient descent. It seems reasonable to assume that for a small λ the global minimum we found should be close, and even if not proven, it seems that f_λ converges to f as $\lim_{\lambda \rightarrow 0^+}$.

Furthermore, $\frac{1}{L}$ in theory and practice on this small scale problem is a good step size, but it can be too conservative as it assures convergence without guaranteeing the fastest rate. With a constant step size, we found the trade off that a larger step size like 10^{-3} converges faster initially but risks overshooting, while a smaller step size like 10^{-6} (closer to the theoretical $\frac{1}{L}$ ensures close proximity to the minimum but converges very slowly. This made it more apparent, the concepts from lecture 3, as for why in industry, step sizes are usually varied dynamically during training (for example line search) rather than using a constant step size, getting the best of both worlds.

We also learned that "vectorizing" our code is an efficient way to do complex computations involving vectors and matrices by avoiding traditional for-loops, which are computationally more costly, by exploiting the CPU capabilities that can process multiple numbers in parallel instead of one at a time. Moreover, it also reduces potential errors related to computer operations.